

Quy Trình Tích Hợp API (Backend → Frontend)

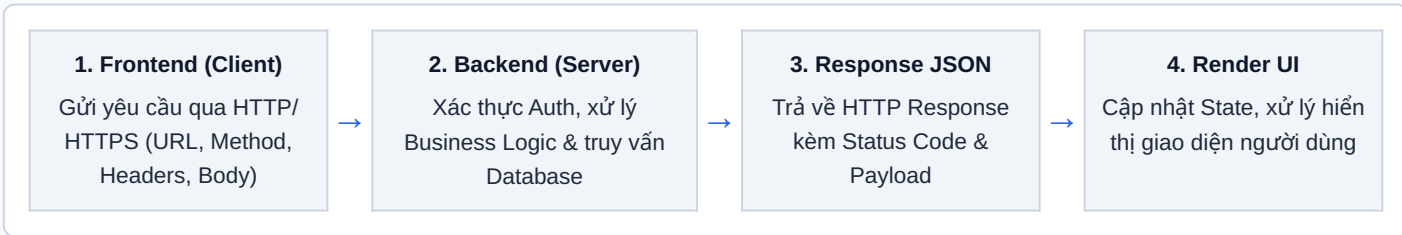
Chuẩn hóa 7 bước mô hình dự án thực tế | Tài liệu kiến trúc & quy chuẩn phát triển

RESTful API

Standard Architecture

Production Ready

1. TỔNG QUAN LUỒNG DỮ LIỆU (DATA FLOW)



2. CHI TIẾT CÁC BƯỚC TÍCH HỢP CHUẨN DỰ ÁN

Bước 1: Chuẩn Hóa & Chuẩn Bị Phía Backend

• Định nghĩa API Specification (Specs):

- Endpoint Path:** Định danh tài nguyên chuẩn REST (ví dụ: `/api/v1/users`).
- HTTP Methods:** **GET** Lấy dữ liệu | **POST** Tạo mới | **PUT / PATCH** Cập nhật | **DELETE** Xóa dữ liệu.
- Headers:** Quy định các thông số chuẩn như `Content-Type: application/json`, `Authorization: Bearer <token>`.
- Parameters / Payload:** Xác định rõ Query Params, Path Variables hoặc Request Body JSON.

- Cấu hình CORS (Cross-Origin Resource Sharing):** Backend bắt buộc phải whitelist Domain/Port của Frontend (Origin Header) để không bị trình duyệt chặn do cơ chế SOP (Same-Origin Policy).

• Chuẩn hóa Response Structure & Status Code:

Mã lỗi (Code)	Ý nghĩa	Cấu trúc JSON Chuẩn
200 / 201	Thành công / Tạo thành công	
400 / 401	Dữ liệu không hợp lệ / Chưa xác thực	{ "status": "success error", "message": "Thông báo ngắn gọn", "data": { ... }, "errors": null }
403 / 404	Không có quyền / Không tìm thấy API	
500	Lỗi hệ thống phía Server	

- Xuất Tài Liệu API (API Docs):** Cung cấp tài liệu rõ ràng thông qua Swagger/OpenAPI hoặc Postman Collection giúp Frontend tra cứu chính xác.

Bước 2: Kiểm Thử Độc Lập API (API Testing)

- Sử dụng các công cụ chuyên dụng: **Postman**, **Insomnia**, hoặc **Bruno**.
- Thực hiện test độc lập tầng Backend trước khi tích hợp vào code Frontend.
- Đảm bảo API trả về đúng HTTP Status Code, Header và cấu trúc JSON đúng cam kết tài liệu.

Bước 3: Quản Lý Cấu Hình & Biến Môi Trường (Frontend)

- **Tuyệt đối không Hardcode URL Server:** Tạo các file môi trường (như `.env.local`, `.env.production`).
- Quản lý Base URL tập trung (ví dụ: `VITE_API_BASE_URL=https://api.example.com`).
- Giúp linh hoạt chuyển đổi giữa môi trường **Local/Development**, **Staging (Testing)**, và **Production**.

Bước 4: Xây Dựng Tầng HTTP Service & Interceptors

- **Tổ chức Kiến trúc Service Layer:** Gom toàn bộ logic gọi API vào từng file Service riêng biệt (`authService`, `userService`, `productService`). Không gọi API rải rác trong UI Components.
- **Request Interceptors (Bộ chặn gửi):** Tự động chèn token xác thực (`Authorization: Bearer <Token>`) vào mọi outgoing request.
- **Response Interceptors (Bộ chặn nhận):** Bắt và xử lý lỗi tập trung:
 - Khi gặp **401 Unauthorized**: Tự động điều hướng về màn hình Login hoặc kích hoạt luồng **Refresh Token**.
 - Khi gặp **500 Internal Error**: Hiển thị thông báo sự cố toàn hệ thống.

Bước 5: Quản Lý Vòng Đời & Trạng Thái Dữ Liệu (UI State Management)

Một luồng gọi API hoàn chỉnh trên Frontend phải quản lý đủ 3 trạng thái bắt buộc:

- **1. Loading State (Đang tải):** Bật cờ trạng thái đang chờ để hiển thị Skeleton Screen hoặc Spinner.
- **2. Success State (Thành công):** Lưu trữ payload nhận được vào State/Store (Redux, Zustand, React State) và render dữ liệu lên UI.
- **3. Error State (Thất bại / Lỗi):** Bắt ngoại lệ qua `try...catch`, hiển thị Toast message hoặc Alert thông báo lỗi thân thiện cho người dùng.

Mẹo nâng cao: Sử dụng các thư viện Data Fetching hiện đại (React Query / TanStack Query, SWR) để tự động hóa việc caching, retry tự động khi lỗi mạng và re-fetch dữ liệu chạy ngầm.

Bước 6: Cơ Chế Xác Thực & Bảo Mật (Authentication & Authorization)

- **Quy trình Đăng nhập:** Frontend gửi Credential (Username/Password) → Backend xác thực → Trả về `Access Token` và `Refresh Token`.
- **Lưu trữ Token an toàn:** Sử dụng **HttpOnly Cookie** do Server set để chống tấn công XSS (hoặc Storage phù hợp tùy bài toán).
- **Ủy quyền (Authorization Header):** Mọi request sau đó được gửi kèm Token ở Header `Authorization`.
- **Cơ chế Refresh Token tự động:** Khi Access Token hết hạn (nhận response 401), Response Interceptor lập tức bắt lấy, âm thầm gửi Refresh Token lên lấy Access Token mới mà người dùng không bị văng ra ngoài.

Bước 7: Kiểm Thử & Debug Kết Nối (DevTools & Edge Cases)

- **Sử dụng Browser DevTools (F12 → Tab Network):**
 - Soi chi tiết request gửi đi: URL, Method, Headers, Request Body / Query Params.
 - Soi phản hồi nhận về: Status Code, Response Headers, JSON Payload.
- **Xử lý đầy đủ các Edge Cases (Trường hợp biên):**
 - Mạng bị gián đoạn / Mất kết nối (Offline state).
 - Request vượt quá thời gian chờ (Timeout).
 - Dữ liệu nhận về rỗng (`null` / `empty list`).

3. TÓM TẮT TRÁCH NHIỆM BACKEND VS FRONTEND

Trách Nhiệm Phía Backend	Trách Nhiệm Phía Frontend
• Định nghĩa API Endpoints & Data Model	• Tạo Service Layer & Cấu hình Base URL
• Xử lý Logic Business, Database & Validation	• Quản lý 3 trạng thái UI (Loading, Success, Error)
• Cấu hình CORS cho phép Frontend truy cập	• Gửi Token ở Request Interceptor
• Trả về đúng HTTP Status Code & chuẩn JSON	• Bắt lỗi tập trung ở Response Interceptor & Refresh Token
• Đóng gói Swagger/Postman API Documentation	• Soi lỗi kỹ thuật trên Browser DevTools Network Tab

⚡ TẬP TRUNG 80/20: 20% KIẾN THỨC NẮM CHẮC 80% THỰC TẾ (DÀNH CHO KHANG)

- **CORS:** 90% lỗi kết nối ban đầu là do Backend quên cho phép Origin của Frontend. Luôn kiểm tra CORS đầu tiên nếu gặp lỗi kết nối.
- **Interceptors (Axios / Fetch Wrapper):** Đừng bao giờ dính kèm Token thủ công ở từng trang. Cấu hình 1 lần duy nhất trong Interceptor để tiết kiệm 80% công sức.
- **3 Trạng thái UI (Loading - Success - Error):** Giao diện tốt luôn hiển thị Skeleton/Spinner khi loading và báo lỗi thân thiện khi fail. Không bao giờ để trang trắng trơn.
- **Refresh Token Flow:** Đây là thước đo phân biệt dev Junior và Mid/Senior. Hiểu rõ luồng âm thầm đổi Token khi hết hạn sẽ giúp trải nghiệm người dùng không bị gián đoạn.